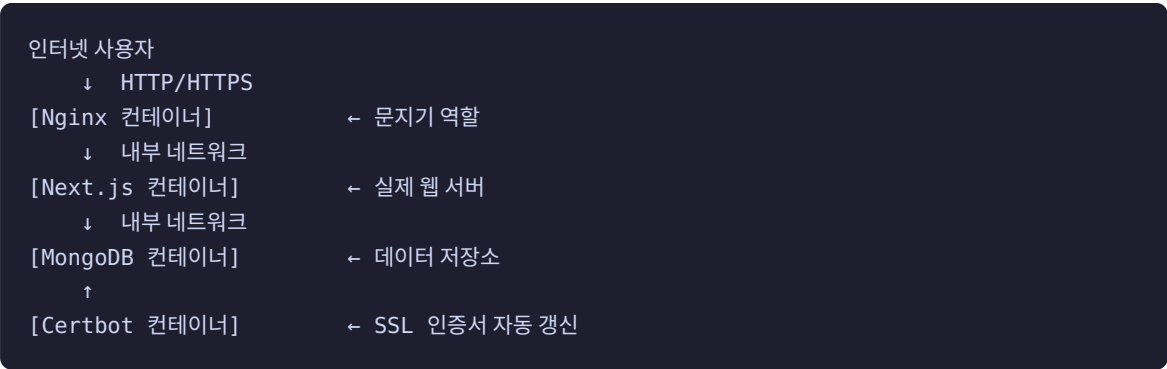


# Ma\_Blog 인프라 코드 학습 가이드

네가 직접 작성한 코드를 기반으로, 각 파일이 왜 이렇게 생겼는지 설명하는 학습 자료야.

## 1. 전체 구조 이해하기

먼저 큰 그림부터 잡자.



이 4개 컨테이너가 OCI 서버 위에서 동시에 돌아가고 있어. Docker Compose가 이 전체를 한 번에 띄우고 관리해줘.

## 2. Dockerfile — Next.js 앱을 컨테이너로 만드는 방법

```
# -----
# Build Stage
# -----
FROM node:20-alpine AS builder
WORKDIR /app

COPY package.json package-lock.json ./
RUN npm ci

COPY . .
RUN npm run build

# -----
# Runner Stage
```

```
# -----
FROM node:20-alpine AS runner

RUN apk add --no-cache docker-cli
RUN addgroup -S nextjs && adduser -S nextjs -G nextjs
RUN addgroup -g 999 ping || true
RUN addgroup nextjs ping

ENV NODE_ENV=production
ENV PORT=8080

WORKDIR /app

COPY --from=builder /app/.next/standalone ./
COPY --from=builder /app/.next/static ./next/static
COPY --from=builder /app/public /public

USER nextjs

EXPOSE 8080
CMD ["node", "server.js"]
```

## 핵심 개념: Multi-Stage Build (다단계 빌드)

이 Dockerfile의 가장 중요한 포인트는 **빌드 단계**와 **실행 단계**를 분리한 것이야.

왜 두 단계로 나누는가?

나쁜 방법 (한 단계로 다 하면):

최종 이미지 크기 = node\_modules(800MB) + 소스코드 + 빌드 결과물

좋은 방법 (Multi-Stage):

Builder 컨테이너: 빌드만 하고 버림  
Runner 컨테이너: 빌드 결과물만 복사 → 최종 이미지 크기 대폭 감소

각 줄 설명

| 코드                                     | 의미                                                           |
|----------------------------------------|--------------------------------------------------------------|
| FROM node:20-alpine AS builder         | node 20 버전 이미지를 기반으로 “builder”라는 이름의 단계 시작. alpine은 경량 Linux |
| WORKDIR /app                           | 이 컨테이너 안에서 작업 디렉토리를 /app으로 설정                                |
| COPY package.json package-lock.json ./ | 소스코드보다 패키지 파일을 먼저 복사 (캐시 최적화)                                |
| RUN npm ci                             |                                                              |

| 코드                                         | 의미                                             |
|--------------------------------------------|------------------------------------------------|
|                                            | npm install과 비슷하지만, lock 파일을 정확히 따름 (CI 환경 전용) |
| <code>COPY . .</code>                      | 나머지 소스코드 전체 복사                                 |
| <code>RUN npm run build</code>             | Next.js 빌드 실행                                  |
| <code>FROM node:20-alpine AS runner</code> | 새 단계 시작. 이전 builder 단계와 완전히 독립된 새 이미지          |
| <code>apk add --no-cache docker-cli</code> | Docker 명령어 설치 (docker ps 실행을 위해 필요)            |
| <code>addgroup -S nextjs</code>            | nextjs 그룹 생성 (-S는 시스템 그룹)                      |
| <code>adduser -S nextjs -G nextjs</code>   | nextjs 유저 생성 (루트로 실행하면 보안 위험)                  |
| <code>addgroup -g 999 ping</code>          | GID 999로 ping 그룹 생성 (docker.sock 접근 권한용)       |
| <code>addgroup nextjs ping</code>          | nextjs 유저를 ping 그룹에 추가 → docker.sock 사용 가능     |
| <code>ENV NODE_ENV=production</code>       | 환경변수 설정. Next.js가 프로덕션 모드로 동작                  |
| <code>COPY --from=builder ...</code>       | builder 단계의 결과물만 복사 (소스코드, node_modules는 제외)   |
| <code>.next/standalone</code>              | Next.js가 standalone 모드로 빌드하면 이 폴더 하나로 실행 가능    |
| <code>USER nextjs</code>                   | 이 줄부터 루트 대신 nextjs 유저로 실행 (보안)                 |
| <code>EXPOSE 8080</code>                   | 이 포트를 사용하겠다는 선언 (실제 열리는 건 아님, 문서 역할)           |
| <code>CMD ["node", "server.js"]</code>     | 컨테이너 시작 시 실행할 명령어                              |

## docker.sock이란?

`/var/run/docker.sock` 은 Docker 데몬과 통신하는 소켓 파일이야. 이걸 컨테이너 안에 마운트하면, 컨테이너 안에서 `docker ps` 같은 명령어를 실행할 수 있어. 블로그 대시보드에 Docker 컨테이너 상태를 보여주는 기능이 이것 때문에 가능한 거야.

△ 보안 주의: docker.sock을 마운트하면 컨테이너가 호스트의 Docker를 제어할 수 있어서 신뢰할 수 있는 컨테이너에만 허용해야 해.

### 3. docker-compose.yml — 4개 컨테이너를 한 번에 관리

```
services:
  db:
    image: mongo:7.0
    container_name: mablog_mongodb
    restart: always
    ports:
      - "127.0.0.1:27017:27017"
    volumes:
      - ./mongo_data:/data/db
      - ./backup:/tmp/backup
    environment:
      MONGO_INITDB_ROOT_USERNAME: mongoAdmin
      MONGO_INITDB_ROOT_PASSWORD: mongoA96547852!
      MONGO_INITDB_DATABASE: maBlog_DataTable
    networks:
      - mablog_network

  mablog_nextjs:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: mablog_nextjs
    restart: always
    expose:
      - '8080'
    env_file:
      - ./env.production
    environment:
      - HOST=0.0.0.0
      - PORT=8080
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock
      - /opt/ma_blog/read_os:/opt/ma_blog/read_os
    depends_on:
      - db
    networks:
      - mablog_network

  mablog_proxy:
    image: nginx:stable-alpine
    container_name: mablog_proxy
    restart: always
    ports:
      - '80:80'
      - '443:443'
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf:ro
      - ./certbot/www:/var/www/certbot
      - ./certbot/conf:/etc/letsencrypt
    depends_on:
      - mablog_nextjs
    networks:
      - mablog_network
```

```

certbot:
  image: certbot/certbot
  container_name: certbot
  restart: unless-stopped
  volumes:
    - ./certbot/www:/var/www/certbot
    - ./certbot/conf:/etc/letsencrypt
  entrypoint: >
    sh -c "trap exit TERM;
    while ;; do certbot renew ...; sleep 12h; done"
  networks:
    - mablog_network

networks:
  mablog_network:
    driver: bridge

```

## 핵심 개념들

### image vs build

```

# image: 이미 만들어진 이미지를 그대로 사용
db:
  image: mongo:7.0

# build: 내 Dockerfile로 이미지를 직접 빌드
mablog_nextjs:
  build:
    context: .
    dockerfile: Dockerfile

```

MongoDB, Nginx는 공식 이미지를 그대로 쓰고, Next.js 앱만 직접 빌드해.

### ports vs expose

```

# ports: 호스트(OCI 서버) ↔ 컨테이너 포트 연결 (외부 접근 가능)
mablog_proxy:
  ports:
    - '80:80'      # 호스트:80 → 컨테이너:80
    - '443:443'   # 호스트:443 → 컨테이너:443

# expose: 컨테이너 간 통신 포트만 열림 (외부 접근 불가)
mablog_nextjs:
  expose:
    - '8080'      # Docker 네트워크 안에서만 8080 접근 가능

```

이 설계 덕분에: - 외부에서는 80, 443 포트만 접근 가능 (Nginx로만 들어옴) - Next.js 8080 포트는 외부에서 직접 접근 불가 (Nginx 통해서만) - MongoDB 27017은 127.0.0.1에만 바인딩 → OCI 서버 자체에서만 접근 가능

## volumes (볼륨) — 데이터 영속성

```
db:
  volumes:
    - ./mongo_data:/data/db    # 호스트 경로:컨테이너 경로
```

컨테이너는 기본적으로 재시작하면 데이터가 날아가. 볼륨으로 호스트 폴더와 연결하면 컨테이너가 사라져도 데이터는 유지돼.

| 볼륨                                           | 역할                         |
|----------------------------------------------|----------------------------|
| <code>./mongo_data:/data/db</code>           | MongoDB 데이터 영구 저장          |
| <code>./certbot/conf:/etc/letsencrypt</code> | SSL 인증서 영구 저장              |
| <code>/var/run/docker.sock</code>            | 호스트 Docker 소켓 공유           |
| <code>./nginx.conf:....:ro</code>            | nginx 설정 파일 읽기 전용(:ro) 마운트 |

## networks (네트워크)

```
networks:
  mablog_network:
    driver: bridge
```

모든 컨테이너가 `mablog_network` 라는 가상 네트워크에 연결돼 있어. 같은 네트워크에 있으면 컨테이너 이름으로 서로를 찾을 수 있어.

```
nginx.conf 안에:
server mablog_nextjs:8080;    ← "mablog_nextjs"가 컨테이너 이름이자 DNS 주소
```

## restart 정책

| 값                           | 동작                      |
|-----------------------------|-------------------------|
| <code>always</code>         | 항상 재시작 (서버 재부팅해도 자동 실행) |
| <code>unless-stopped</code> | 수동으로 멈추지 않는 한 항상 재시작    |
| <code>on-failure</code>     | 에러로 종료될 때만 재시작          |

## depends\_on — 실행 순서 보장

```
mablog_nextjs:
  depends_on:
    - db    # db 컨테이너가 먼저 시작된 후에 nextjs 시작
```

```
mablog_proxy:
  depends_on:
    - mablog_nextjs # nextjs가 먼저 시작된 후에 nginx 시작
```

△ 주의: depends\_on은 “시작 순서”만 보장해. 컨테이너가 “완전히 준비됐는지”는 보장 안 해. MongoDB가 실제로 접속 가능한 상태가 될 때까지 기다리지 않아.

## Certbot — SSL 인증서 자동 갱신

```
certbot:
  entrypoint: >
    sh -c "trap exit TERM;
    while ;; do
      certbot renew --webroot -w /var/www/certbot;
      sleep 12h;
    done"
```

이 컨테이너는 12시간마다 Let's Encrypt 인증서 갱신을 시도해. `trap exit TERM` 은 컨테이너가 종료 신호를 받으면 깔끔하게 종료되도록 하는 거야.

## 4. nginx.conf — 트래픽 라우팅과 HTTPS

```
events {}

http {
    log_format custom_log '$remote_addr - $remote_user [$time_local] '
                          '"$request" $status $body_bytes_sent '
                          '"$http_referer" "$http_user_agent" '
                          'RT=$request_time';

    access_log /dev/stdout custom_log;
    error_log /dev/stderr warn;

    upstream nextjs_upstream {
        server mablog_nextjs:8080;
    }

    # HTTP → HTTPS 리다이렉트
    server {
        listen 80;
        server_name madayblog.com www.madayblog.com;

        location ^~ /.well-known/acme-challenge/ {
            root /var/www/certbot;
            default_type "text/plain";
        }
    }
}
```

```

        location / {
            return 301 https://$host$request_uri;
        }
    }

    # HTTPS 서버
    server {
        listen 443 ssl http2;
        server_name madayblog.com www.madayblog.com;

        ssl_certificate /etc/letsencrypt/live/madayblog.com/fullchain.pem;
        ssl_certificate_key /etc/letsencrypt/live/madayblog.com/privkey.pem;
        ssl_protocols TLSv1.2 TLSv1.3;

        location ^~ /.well-known/acme-challenge/ {
            root /var/www/certbot;
        }

        location / {
            proxy_pass http://nextjs_upstream;
            proxy_http_version 1.1;
            proxy_set_header Upgrade $http_upgrade;
            proxy_set_header Connection "upgrade";
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
            proxy_set_header X-Forwarded-Proto https;
        }

        location /api {
            proxy_pass http://nextjs_upstream;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
            proxy_set_header X-Forwarded-Proto https;
        }
    }
}

```

## 핵심 개념들

### Nginx의 역할: Reverse Proxy

사용자 브라우저 → Nginx → Next.js

사용자는 Nginx와만 통신해. Next.js는 직접 노출되지 않아. 이것 **리버스 프록시**라고 해.

#### 왜 이렇게 하는가?

- SSL 처리를 Nginx 한 곳에서 담당 (Next.js는 SSL 신경 안 써도 됨)
- 여러 서비스(API 서버, 정적 파일 서버 등)를 하나의 도메인으로 묶을 수 있음



- 보안: 내부 서버 구조를 외부에 노출하지 않음

## upstream

```
upstream nextjs_upstream {
    server mablog_nextjs:8080;
}
```

`nextjs_upstream` 이라는 이름으로 Next.js 서버를 묶어놓은 거야. 나중에 서버를 여러 대로 늘릴 때 여기에 추가하면 로드 밸런싱이 자동으로 돼.

## HTTP → HTTPS 강제 리다이렉트

```
server {
    listen 80;          # HTTP 포트
    location / {
        return 301 https://$host$request_uri;
    }
}
```

`http://madayblog.com` 으로 접속하면 자동으로 `https://madayblog.com` 으로 보내줘. 301은 영구 리다이렉트 HTTP 상태 코드야.

## ACME Challenge — Let's Encrypt 인증 방식

```
location ^~ /.well-known/acme-challenge/ {
    root /var/www/certbot;
}
```

Let's Encrypt가 “이 서버가 정말 madayblog.com 소유자인가?” 확인할 때 쓰는 경로야. Certbot이 이 폴더에 파일을 만들고, Let's Encrypt 서버가 접근해서 확인해.

`^~` 는 이 경로는 다른 규칙보다 우선으로 처리하라는 뜻이야. HTTP 서버에서도 이 경로를 열어둬야 갱신 시 리다이렉트에 막히지 않아.

## SSL 설정

```
ssl_certificate      /etc/letsencrypt/live/madayblog.com/fullchain.pem;
ssl_certificate_key  /etc/letsencrypt/live/madayblog.com/privkey.pem;
ssl_protocols        TLSv1.2 TLSv1.3;
ssl_prefer_server_ciphers off;
```

| 항목                         | 설명                           |
|----------------------------|------------------------------|
| <code>fullchain.pem</code> | 내 인증서 + 중간 인증서 체인 (브라우저에 전달) |

| 항목                                         | 설명                                |
|--------------------------------------------|-----------------------------------|
| <code>privkey.pem</code>                   | 개인 키 (절대 외부에 노출되면 안 됨)            |
| <code>TLSv1.2 TLSv1.3</code>               | 오래된 TLS 1.0, 1.1은 보안 취약점으로 차단     |
| <code>ssl_prefer_server_ciphers off</code> | 클라이언트가 선호하는 암호화 방식 사용 (현대적 권장 설정) |

### proxy\_set\_header — 요청 정보 전달

```
proxy_set_header Host $host;
proxy_set_header X-Real-IP $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto https;
```

Nginx가 Next.js로 요청을 전달할 때, 원본 요청 정보도 같이 넘겨줘. 없으면 Next.js 입장에서 “모든 접속이 Nginx에서 오는 것처럼” 보여.

| 헤더                             | 의미                         |
|--------------------------------|----------------------------|
| <code>Host</code>              | 원래 요청한 도메인 (madayblog.com) |
| <code>X-Real-IP</code>         | 실제 사용자 IP                  |
| <code>X-Forwarded-For</code>   | 프록시를 거친 IP 체인              |
| <code>X-Forwarded-Proto</code> | 원래 프로토콜 (https)            |

### WebSocket 지원

```
proxy_http_version 1.1;
proxy_set_header Upgrade $http_upgrade;
proxy_set_header Connection "upgrade";
```

Next.js의 HMR(개발 중 핫 리로드)이나 WebSocket 연결을 위해 필요해. HTTP 1.1부터 WebSocket 업 그레이드가 가능해.

## 5. server/route.ts — 서버 리소스 모니터링

```
let lastTotal = 0;
let lastIdle = 0;

function ReadCPU() {
  const stat = fs.readFileSync('/proc/stat', 'utf8');
  const line = stat.split('\n')[0]; // "cpu 3357 0 4313 ..."
}
```

```

const fields = line.split(/\s+/).slice(1).map(Number);
const idle = fields[3];
const total = fields.reduce((a, b) => a + b, 0);
return { idle, total };
}

function GetCPUUsage() {
  const { idle, total } = ReadCPU();
  const idleDelta = idle - lastIdle;
  const totalDelta = total - lastTotal;
  lastTotal = total;
  lastIdle = idle;
  const usage = 100 * (1 - idleDelta / totalDelta);
  return Math.max(0, Math.min(100, usage));
}

```

## /proc/stat 이란?

Linux에는 `/proc` 라는 특별한 폴더가 있어. 파일처럼 생겼지만 실제 파일이 아니고, 커널이 실시간으로 만들어주는 가상 파일이야.

```

/proc/stat 예시:
cpu 10132 0 8712 194726 332 0 1234 0 0 0
   user  nice  system idle  iowait ...

```

각 숫자는 CPU가 특정 작업에 쓴 “tick” 수야.

### CPU 사용률 계산 원리

순간 스냅샷 하나로는 CPU 사용률을 계산할 수 없어. 두 번 측정해서 차이를 계산해야 해.

```

CPU 사용률 = 1 - (idle 증가량 / 전체 증가량)
            = 100 * (totalDelta - idleDelta) / totalDelta

```

`lastTotal`, `lastIdle` 이 전역 변수인 이유가 이거야 — 이전 측정값을 기억해야 해.

### 외부 쉘 스크립트 실행

```

const { stdout } = await execAsync("sh /opt/ma_blog/read_os/status.sh");
const data: ServerType = JSON.parse(stdout);

```

메모리, 디스크 정보는 Node.js로 읽기 어렵기 때문에 쉘 스크립트를 별도로 만들어서 실행하고 JSON으로 파싱해.

---

## 6. docker/route.ts — SSE로 Docker 상태 실시간 전송

```
export async function GET(request: Request) {
  const encoder = new TextEncoder();

  const stream = new ReadableStream({
    async start(controller) {
      const send = async () => {
        const containers = await getContainers();
        controller.enqueue(
          encoder.encode(`data: ${JSON.stringify(containers)}\n\n`)
        );
      };

      await send();
      const interval = setInterval(send, 5000);

      request.signal.addEventListener('abort', () => {
        clearInterval(interval);
        controller.close();
      });
    },
  });

  return new Response(stream, {
    headers: {
      'Content-Type': 'text/event-stream',
      'Cache-Control': 'no-cache',
      'Connection': 'keep-alive',
    },
  });
}
```

### SSE (Server-Sent Events) 란?

일반 HTTP 요청은 “요청 → 응답 → 연결 종료” 방식이야. SSE는 연결을 끊지 않고 서버가 계속 데이터를 보내는 방식이야.

클라이언트 ————— 연결 요청 —————> 서버  
클라이언트 <— data: [...] — 5초마다 전송 — 서버

**vs WebSocket:** - WebSocket: 양방향 통신 (클라이언트도 서버에 보낼 수 있음) - SSE: 단방향 (서버 → 클라이언트만). 더 단순하고 HTTP 기반

### SSE 데이터 형식

```
data: {"ID":"abc123","Names":"mablog_nextjs"}\n\n
```

반드시 `data:` 로 시작하고 `\n\n` (빈 줄 2개)으로 끝나야 해. 이게 SSE 프로토콜 규칙이야.

### 메모리 누수 방지

```
request.signal.addEventListener('abort', () => {  
  clearInterval(interval); // 5초 타이머 정리  
  controller.close();      // 스트림 닫기  
});
```

브라우저가 페이지를 떠나면 `abort` 이벤트가 발생해. 이때 타이머를 정리하지 않으면 아무도 안 보는 곳에 5초마다 계속 데이터를 전송해. 반드시 이렇게 정리해줘야 해.

### docker ps 명령어 실행

```
exec("docker ps --format '{{json .}}'", (err, stdout) => {  
  const list = stdout.trim().split('\n').map(line => JSON.parse(line));  
});
```

`--format '{{json .}}'` 옵션으로 각 컨테이너 정보를 JSON 형태로 받아. 한 줄 = 컨테이너 1개이므로 줄바꿈으로 분리해서 파싱해.

## 7. 전체 요청 흐름 정리

브라우저가 <https://madayblog.com> 에 접속할 때 어떤 일이 일어나는지 따라가보자.

1. 브라우저가 DNS 조회 → OCI 서버 IP 획득
2. 브라우저 → OCI 서버:443 (HTTPS 연결)
3. Nginx 컨테이너 수신
  - SSL 인증서로 암호화 해제
  - 요청 헤더에 X-Real-IP, X-Forwarded-For 추가
  - Docker 내부 네트워크로 Next.js 컨테이너에 전달
4. Next.js 컨테이너 수신
  - 페이지 컴포넌트 실행 (서버 컴포넌트)
  - MongoDB 컨테이너에 데이터 조회 (같은 Docker 네트워크)
  - HTML 생성 후 Nginx에 반환
5. Nginx → 브라우저에 응답
6. 브라우저가 `/api/system/docker` 에 SSE 연결 요청
  - Nginx가 Next.js로 프록시

- Next.js가 docker ps 실행 후 5초마다 결과 전송
- 브라우저가 실시간으로 컨테이너 상태 업데이트

---

## 8. 자주 쓰는 명령어 정리

---

```
# 모든 컨테이너 내리기
docker-compose down

# 빌드 포함해서 올리기 (코드 변경 후 반드시 --build)
docker-compose up -d --build

# 특정 컨테이너 로그 보기
docker logs mablog_nextjs -f

# 실행 중인 컨테이너 목록
docker ps

# 컨테이너 안에 접속
docker exec -it mablog_nextjs sh

# Nginx 설정 테스트 (문법 검사)
docker exec mablog_proxy nginx -t

# Nginx 설정 리로드 (재시작 없이)
docker exec mablog_proxy nginx -s reload
```

---

이 문서는 madayblog.com 실제 코드 기반으로 작성됨